WEEK 0 : PRE-REQUISITE CONCEPTS TO BUILD A KNOWLEDGE GRAPH RAG

1: Asynchronous Programming & Non-Blocking I/O Architecture

- **The Traditional Synchronous mechanism -** lets consider a kitchen , with only one chef. When someone orders a steak, he stands by the stove and stares at it for 20 minutes doing nothing else. Customers starve because he can only cook one meal at a time.
- **The Modern Asynchronous (Non-Blocking Architecture):**
  - There is **one Head Chef (The Event Loop)**.
  - When a steak goes in the oven, the chef doesn't stare at it. He drops a **coroutine bookmark** on the oven timer, turns around, and chops veggies for the next order.When the oven timer dings (the I/O operation finishes), a helper bell rings.The Head Chef pauses his veggie chopping, plates the steak, and goes right back to chopping.

When you put all of this together – **Event Loops, Non-Blocking I/O, and Coroutines**, you get a high-speed machine that can handle millions of users at the exact same time .

HOW Python's superpower engine, `asyncio`, works under the hood ?

1. OS Spies (`epoll` and `kqueue`)- Event Loop manager is only single-threaded. When it has to wait for 100 different websites to send back data, it doesn't have time to stand at every single door.Instead, it hires Operating System Selectors: * On Linux computers, **epoll** * On Mac computers, **kqueue**

2. The 5 States of a Task

PENDING (**Waiting at the Starting Line**):- asyncio.create_task. …waiting for the Event Loop to notice it.

RUNNING (**Active Action**): The Event Loop picks it up and runs its instructions step-by-step, until it hits an `await` word.

SUSPENDED (Taking a Nap): The moment it hits `await`, it pauses, hands its bookmark to the OS selector(`epoll`/`kqueue`), and goes to sleep.(uses 0% of the computer's brainpower (CPU).)

RESUMPTION (Waking Up): The Event Loop wakes the task up, moves it back to the running line, and it starts right after the `await` word with its brand-new data.

FINISHED / CANCELLED(The End): The task finishes its job and returns an answer, blows up with an error, or gets canceled by the user.

3. The Dangerous Trap: Forgetting the `await` Word - When you write an `async def` function, calling it doesn't actually run the code inside it , When you forget the `await` and just write `fetch_entity_metadata("....")' instead of writing `await fetch_entity_metadata("...")` , Python just hands you the `<coroutine object>`, telling you: "Hey, you asked me to create this task, but you never told me to actually run it or wait for it!"

Python will even throw a runtime warning saying something like: *RuntimeWarning: coroutine 'fetch_entity_metadata' was never awaited.

Understanding Concurrency vs Parallelism -

Concurrency (Juggling): This is about **structure**. It is the ability to *deal with* a lot of different things at once by switching between them quickly.

Parallelism (Cloning / Multiple Workers): This is about **execution**. It is the ability to *do* a lot of different things at the exact **same physical instant**, which requires multiple computer brains (CPU cores).

Why Concurrency Wins for I/O-Bound Engines

You might think: *"If parallelism means using multiple CPU cores, why doesn't every high-throughput engine just use massive parallelism?"* **it just creates traffic jams.**

The Concurrency Solution (asyncio): Instead, you use a **single-threaded event loop (Concurrency)**. One smart manager handles all network connections using non-blocking I/O and coroutine suspension. Because the computer never wastes time switching between heavy threads, it can fly through thousands of requests effortlessly using almost zero RAM and CPU.

Why Parallelism Wins for CPU-Bound Tasks

When it's time to do heavy math like converting thousands of text chunks into vector embeddings or calculating similarity scores , here juggling won't save you. You need **Parallelism (Cloning) ,** handing out to multiple CPU cores or a GPU so they can all crunch numbers at the exact same physical millisecond.

In a real-world RAG system,you use **both** working together like a well-oiled machine:

- Use **Concurrency (**asyncio**)** to fetch your documents, talk to db, and query remote APIs without freezing up.
- Use **Parallelism (GPUs / Multiprocessing)** to crunch all heavy AI embeddings and vector math at lightning speed.

Concurrency handles the traffic and waiting; Parallelism handles the heavy lifting

*grouping tasks into asyncio.gather(*tasks) registers all coroutines concurrently within the event loop's active state machine. Total wall-clock execution time compresses to the duration of the slowest network call:* $O\left(\max_i (t_i)\right)$

{{** For three remote embedding calls taking* $180ms$, $220ms$, *and* $190ms$ *respectively, sequential execution consumes* $\sum t_i = 590ms$. *Concurrent execution via asyncio.gather() completes in* $\max(t_i) = 220ms$. ***}]

Connection Pooling with aiohttp.ClientSession and TCP/SSL Optimization

*Imagine making a web request is like calling a restaurant; dialing, saying hello, and proving your identity takes 3 annoying steps every time -**a Handshake (3 RTTs)** . Only after those **3 RTTs** are finished can you finally ask for your data,* that handshake penalty multiplies into a massive traffic jam. We also make a mistake by wrapping aiohttp.ClientSession() inside a tiny function, now - you are opening a connection, doing the exhausting 3-step handshake, getting your data, and then immediately **hanging up** when the function ends…..That is brutally slow.

Solution - Connection Pooling (ClientSession**)** Instead of hanging up after every single call, **Connection Pooling** means you keep *one* phone line open with a group of friends who stay on the line.

The Three-Way Handshake

1. **SYN (Synchronize):** The client socket transitions from CLOSED to SYN_SENT. It transmits a TCP segment with the SYN flag set, containing an initial sequence number (ISN_c)
2. **SYN-ACK (Synchronize-Acknowledge):** The server receives the packet, allocates memory for the Transmission Control Block, transitions to SYN_RECEIVED, and responds with a segment containing both SYN and ACK flags.
3. **ACK (Acknowledge):** The client receives the packet, transitions to ESTABLISHED, and returns an ACK packet acknowledging $ISN_s + 1$. The server receives this and transitions to ESTABLISHED.

The Kernel Penalty: Port Exhaustion and TIME_WAIT

When an unpooled application destroys a session immediately after use, it invokes a socket teardown, client-side socket enters the TIME_WAIT state (lasting $2 \times MSL$, typically 60 seconds) to ensure delayed packets from the old session evaporate safely.

The Consequence: Constantly opening and closing sockets consumes finite **ephemeral ports** . At high throughput, you hit port exhaustion, alongside massive CPU overhead from system calls and kernel context switches.

Maintaining a single, long-lived `aiohttp.ClientSession` across the application lifecycle enables persistent TCP connection pooling. When sequential or concurrent requests target identical hosts, `ClientSession` checks its pool for active, idle sockets. Reusing warm sockets bypasses TCP and TLS handshakes entirely, eliminating the 3-RTT latency penalty for batch embedding generation and knowledge graph extraction

Rate Limiting, Resilience, and Fault-Tolerant Execution Patterns

High-throughput distributed engines must implement concurrency limits, deadlines, failure isolation, and retry mechanisms to maintain system stability.

1. **Concurrency Throttling (asyncio.Semaphore)**: `SEMAPHORE = asyncio.Semaphore(5)` caps concurrent outgoing network requests to 5. Tasks exceeding this limit pause in the event loop queue without spin-locking or consuming CPU cycles.

2. **Deadline Enforcement (asyncio.wait_for)**: Wrapping execution in `asyncio.wait_for(..., timeout=5.0)` enforces strict deadlines. Sockets that hang trigger an `asyncio.TimeoutError` and cancel the underlying task, executing cleanup via `try...finally` logic.
3. **Fault Isolation (return_exceptions=True)**: Passing `return_exceptions=True` to `asyncio.gather()` prevents single-task exceptions from halting the entire batch. Successful payload processing continues, while exception instances are returned in the result array for handling via `isinstance(r, Exception)`.
4. **Transient vs. Permanent Error Retries with Jitter**: Client errors (e.g., HTTP 401, 404) represent permanent failures and fail fast without retrying. Transient server errors (HTTP 503) or connection drops trigger exponential backoff retries. The delay formula includes randomized jitter: $t_{wait} = 2^{attempt} + random_jitter$ {{ Adding jitter prevents "thundering herd" scenarios, where concurrent worker nodes retry failed requests simultaneously and overwhelm recovering remote servers. }}
5. **Non-Blocking Persistence (aiosqlite)**: `aiosqlite` offloads blocking SQLite disk I/O to an asynchronous thread pool. This allows fetched entity payloads to be written to disk without blocking the main event loop thread.

Mastering non-blocking I/O establishes the high-throughput network substrate required for executing high-dimensional vector math without pipeline bottlenecks,

2: High-Dimensional Vector Math & Linear Algebra Fundamentals

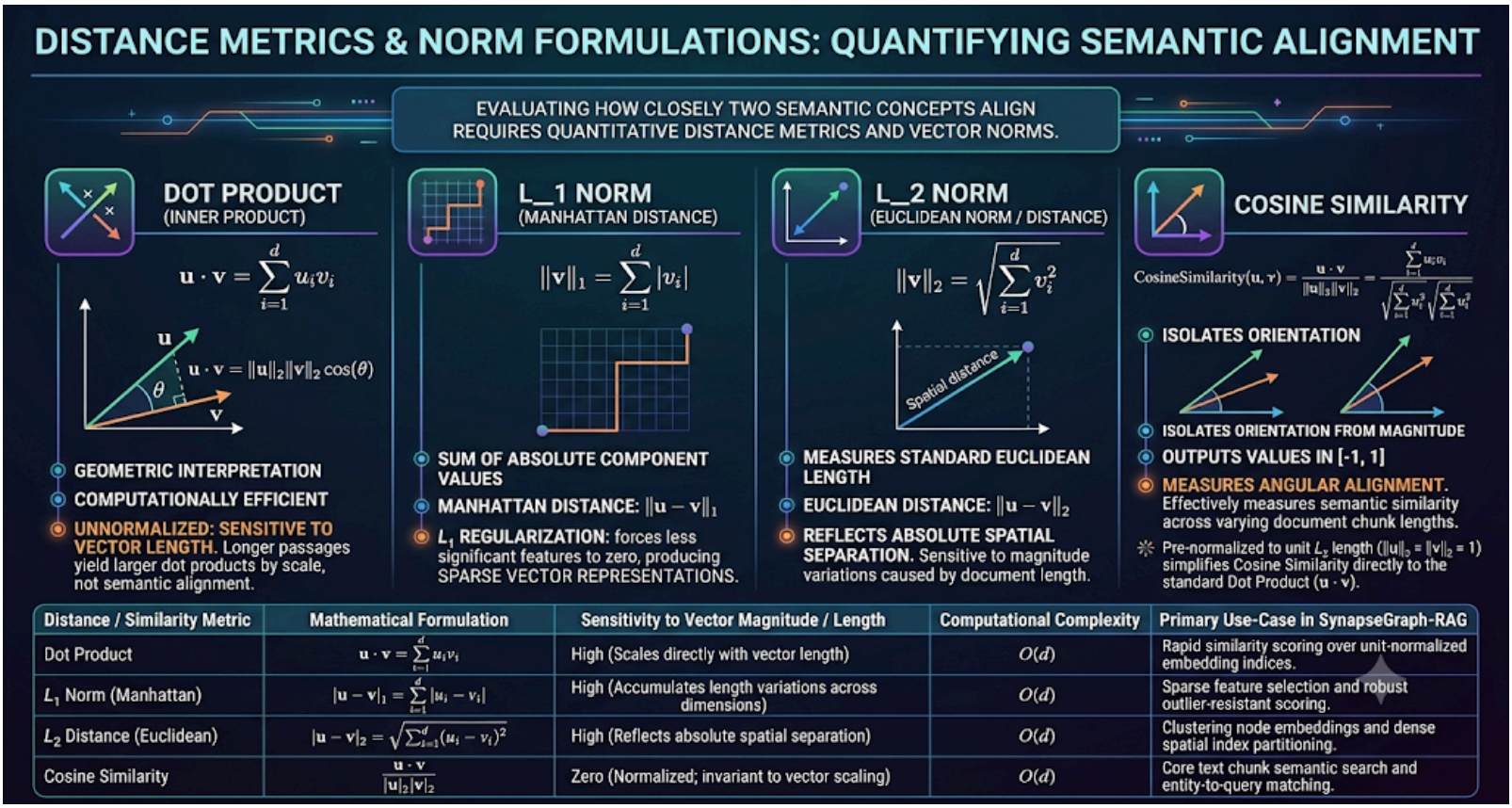
Is the foundation of modern Information Retrieval (IR). Dense continuous vector representations map raw textual data into geometric spaces, enabling semantic indexing, node embedding analysis, and fast similarity search in our SynapseGraph-RAG system.

Geometric Vector Space Representations and Coordinate Systems

the core math behind **how text turns into numbers that computers can search semantically**.

1. **a continuous mathematical manifold** -> **High-Dimensional Space (R^d):** Computers don't understand words like "apple" or "invoice." Instead, models like all-MiniLM-L6-v2 translate every piece of text into a giant list of numbers (a coordinate) with a specific dimension (for MiniLM, $d = 384$). Imagine a map with 384 directions instead of just North, South, East, and West.
2. **Semantic Geometry (Proximity = Meaning):** In this space, geometry mirrors meaning. Sentences that mean the same thing end up as points sitting right next to each other. Unrelated sentences end up miles apart.
3. **Embedding Models as Projection Functions:** The model acts as a translator. It takes messy, discrete human language (tokens) and *projects* it into continuous, smooth coordinates {{ remember that a 384-dimensional vector is just a Python list of 384 floating-point numbers. You should know how changing models changes the dimension size (e.g., OpenAI's text-embedding-3-small uses 1536 dimensions)}
    - **Linear Algebra & Basis Vectors:** every dimension in that 384-element list acts like an independent axis (like X, Y, Z axes, but scaled up to hundreds of directions). By combining these axes, the model can represent nuanced shades of meaning, allowing the computer to perform fast linear algebra (like calculating angles and distances) instead of slow, exact-word matching.
4. **Vector Databases (The Practical Storage):**how tools like FAISS, Chroma, or Pinecone store these high-dimensional points and index them so searching through millions of vectors takes milliseconds instead of seconds -> tools use **Approximate Nearest Neighbor (ANN)** algorithms. Instead of searching everything, they use clever mapping shortcuts to find the right answer 99% of the time, instantly. By organizing high-dimensional space into **navigable graphs (HNSW)** or **clustered regions (IVF)** {{ when a user asks a question, RAG converts the question into a vector and asks the vector database: *"Which stored document vectors have the highest cosine similarity to my question?" }}

Distance Metrics & Norm Formulations



Distance / Similarity Metric	Mathematical Formulation	Sensitivity to Vector Magnitude / Length	Computational Complexity	Primary Use-Case in SynapseGraph-RAG
Dot Product	$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{d} u_i v_i$	High (Scales directly with vector length)	$O(d)$	Rapid similarity scoring over unit-normalized embedding indices.
L_1 Norm (Manhattan)	$\|\mathbf{u} - \mathbf{v}\|_1 = \sum_{i=1}^{d} \|u_i - v_i\|$	High (Accumulates length variations across dimensions)	$O(d)$	Sparse feature selection and robust outlier-resistant scoring.
L_2 Distance (Euclidean)	$\|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}$	High (Reflects absolute spatial separation)	$O(d)$	Clustering node embeddings and dense spatial index partitioning.
Cosine Similarity	$\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$	Zero (Normalized; invariant to vector scaling)	$O(d)$	Core text chunk semantic search and entity-to-query matching.

Matrix Transformations, Linear Mappings, and High-Dimensional Operations

Linear Mapping: When you multiply a matrix $A \in R^{m \times d}$ by a vector $x \in R^{d}$, you perform a linear transformation: $Ax = b \ where \ b \in R^{m}$. This operation acts as a geometric engine. It can rotate, scale, shear, or project the input vector space from one dimensionality (d) into another (m).

This exact mathematical operation is the **heartbeat of the retrieval phase in our RAG pipeline:**

{{ Context Retrieval: The system extracts the top-scoring document chunks (those with similarity scores closest to 1.0 *) and injects them as context into a Large Language Model (LLM), enabling accurate, grounded responses}}*

When solving linear systems of the form $Ax = b$ for vector space projection matrices, computing direct matrix inversions **($A^{-1}b$) is an anti-pattern** {*severe numerical instability -> floating-point rounding errors ; consumes* $O(d^3)$}
High-performance numerical engines use direct matrix factorizations:

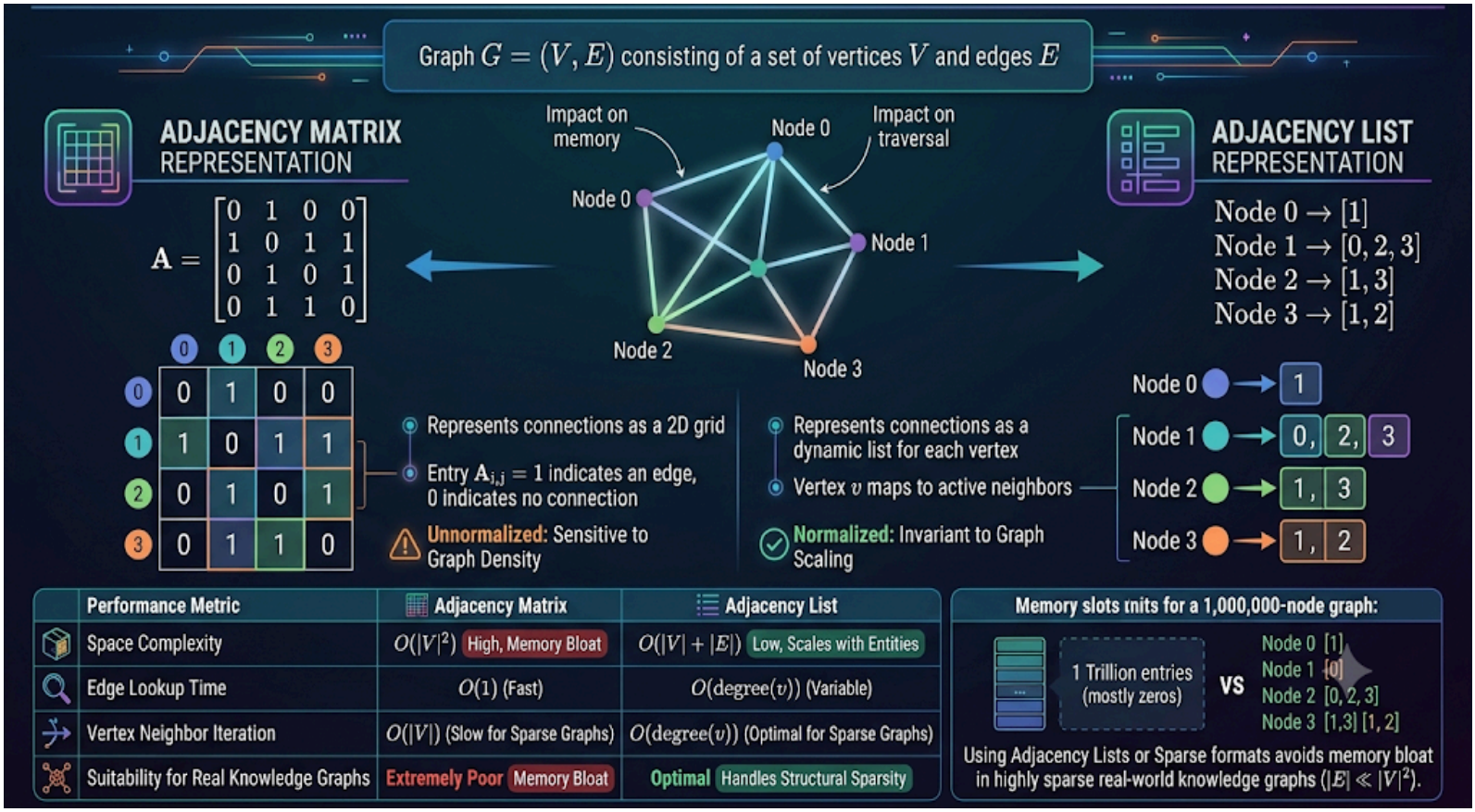
- **LU Decomposition with Partial Pivoting:** Decomposes square matrices into lower and upper triangular systems , **Partial pivoting** introduces a permutation matrix P to reorder rows, ensuring that the largest available element is chosen as the pivot at each step. This guarantees numerical stability.
- **QR Factorization**: Factors rectangular matrices into orthogonal and upper triangular components ($A = QR$) using Gram-Schmidt or Householder transformations.
- **Cholesky Factorization**: Efficient $O(\frac{1}{3}d^3)$ factorization for symmetric +ve-definite covariance matrices ($A = LL^T$) .Computing the inverse or determinant of a massive covariance matrix relies entirely on Cholesky factors for numerical stability.

Vector representations and metric spaces provide the mathematical framework required to compute structural relationships over graph topologies

3: Graph Data Structures, Topology, and NetworkX Traversal Mechanics

While high-dimensional vector spaces handle localized semantic text matching, knowledge graphs structure domain entities and their explicit relationships as relational networks. Graph structures allow our SynapseGraph-RAG engines to execute multi-hop reasoning and extract broader context missed by flat vector databases.

Graph Representation Complexity: Adjacency Matrix vs. Adjacency List



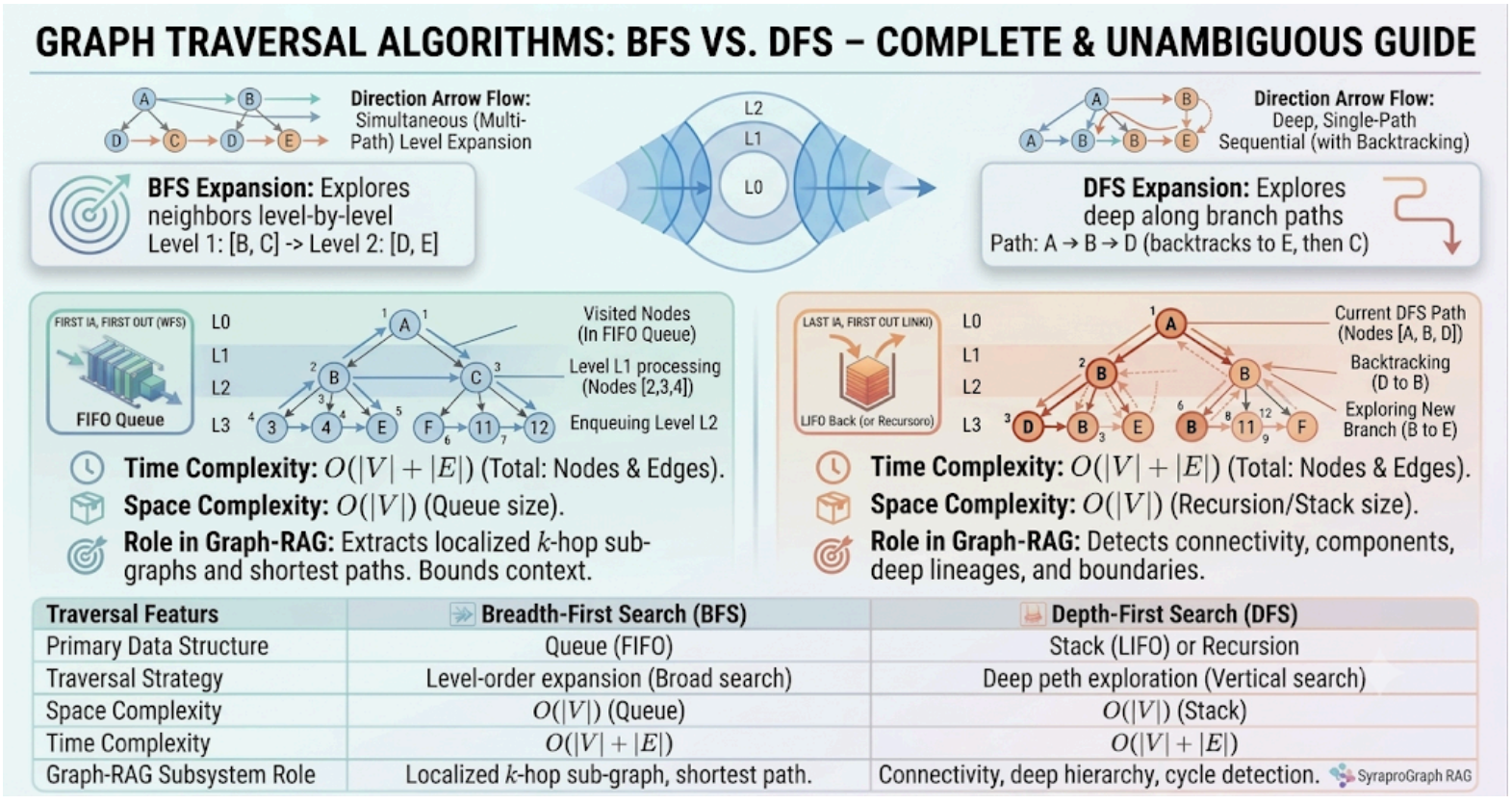
Performance Metric	Adjacency Matrix	Adjacency List
Space Complexity	$O(\|V\|^2)$ High, Memory Bloat	$O(\|V\| + \|E\|)$ Low, Scales with Entities
Edge Lookup Time	$O(1)$ (Fast)	$O(\text{degree}(v))$ (Variable)
Vertex Neighbor Iteration	$O(\|V\|)$ (Slow for Sparse Graphs)	$O(\text{degree}(v))$ (Optimal for Sparse Graphs)
Suitability for Real Knowledge Graphs	Extremely Poor Memory Bloat	Optimal Handles Structural Sparsity

Memory slots units for a 1,000,000-node graph:

1 Trillion entries (mostly zeros) VS Node 0 [1], Node 1 [0], Node 2 [0, 2, 3], Node 3 [1,3] [1, 2]

Using Adjacency Lists or Sparse formats avoids memory bloat in highly sparse real-world knowledge graphs ($|E| \ll |V|^2$).

Graph Traversal Algorithms: Breadth-First Search (BFS) vs. Depth-First Search (DFS)

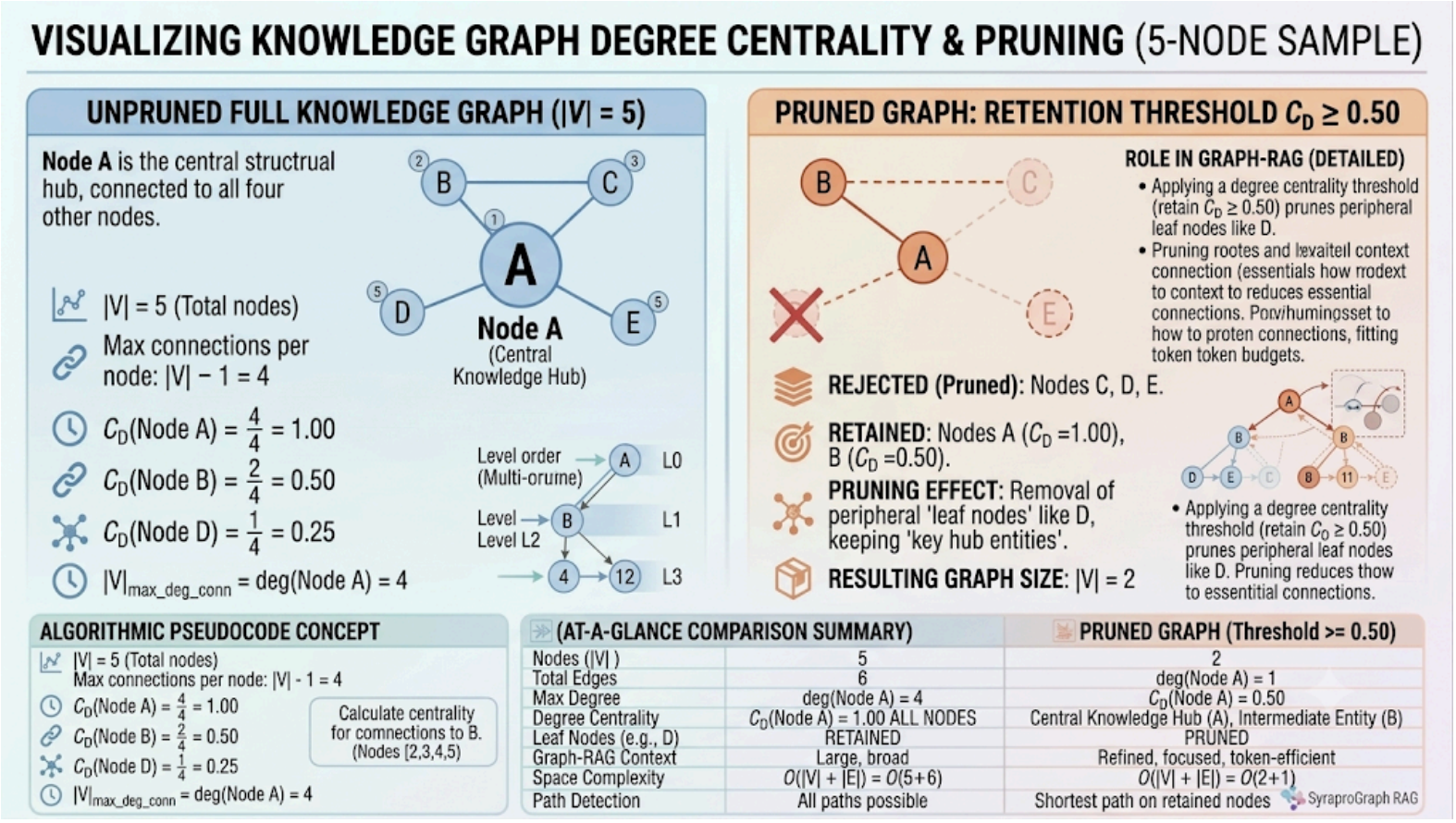


Traversal Features	Breadth-First Search (BFS)	Depth-First Search (DFS)
Primary Data Structure	Queue (FIFO)	Stack (LIFO) or Recursion
Traversal Strategy	Level-order expansion (Broad search)	Deep peth exploration (Vertical search)
Space Complexity	$O(\|V\|)$ (Queue)	$O(\|V\|)$ (Stack)
Time Complexity	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$
Graph-RAG Subsystem Role	Localized k-hop sub-graph, shortest path.	Connectivity, deep hierarchy, cycle detection. SyraproGraph RAG

Topological Centrality Analysis and NetworkX Integration

To prevent token limits from being exceeded when injecting knowledge graph contexts into LLMs, sub-graphs must be pruned based on structural importance. Degree Centrality quantifies a node's local connectivity relative to the total graph size.For an undirected graph, Degree Centrality $C_D(v)$ for a node v is calculated as: $C_D(v) = \frac{deg(v)}{|V|-1}$

where $deg(v)$ represents the number of direct edges connected to node v, and $|V| - 1$ represents the maximum possible connections.



Using Python's standard **graphlib.TopologicalSorter** enables robust Directed Acyclic Graph **(DAG) task scheduling**. It resolves pipeline execution orders for multi-hop graph retrieval, ensuring prerequisite operations complete before downstream attention reranking executes.

Structural graph topologies are merged with high-dimensional contextual representations through attention mechanisms

4: Token Embeddings & Transformer Scaled Dot-Product Attention

Attention mechanisms allow transformer models to contextualize token representations dynamically it enables the system to weight retrieved knowledge graph entities and query tokens based on mutual relevance.

Token Embedding Mappings and Dimensional Projection

Token Embedding Mappings act as the crucial translation layer that turns discrete words or subwords into a continuous geometric space where semantics can be calculated mathematically. An input text sequence is first tokenized into discrete integer IDs.A sentence becomes a sequence of integers .These IDs serve as indices to look up dense representations in a pre-trained embedding matrix: $E \in R^{|V_{vocab}| \times d_{model}}$. When a token has integer ID i, the model extracts the i-th row of matrix E. This transforms a sparse, discrete index into a dense, continuous vector: $x_i \in R^{384}$.

Mathematically, multiplying a one-hot vector representation of a token $v \in R^V$ by the embedding matrix E yields the exact same result: $x = v^T E$.

Pre-trained embedding spaces capture semantic relationships, positioning syntactically or conceptually similar words close to each other prior to sequence contextualization.

Once inside the transformer network, subsequent layers (like self-attention) will shift these coordinates based on the surrounding context (e.g., distinguishing between the word "bank" in a financial context versus a riverbank)

Query, Key, and Value Matrix Projections

While static embeddings give every word a single fixed coordinate space, words change their meaning based on their neighbors. **Query, Key, and Value (Q, K, V) projections** use learned linear transformations to map the input sequence into three distinct "perspectives," allowing tokens to talk to and query one another.

the sequence matrix $X \in R^{N \times d_{model}}$ (N=sequence length) is projected into Query (Q), Key (K), and Value (V) matrices optimizing them for similarity searching. Instead of using X directly for comparison, the model multiplies it by three separate, trainable weight matrices (W_Q, W_K, and W_V):

- **Query Matrix (Q):** $Q = XW_Q where W_Q \in R^{d_{model} \times d_k}$
- **Key Matrix (K):** $K = XW_K where W_K \in R^{d_{model} \times d_k}$
- **Value Matrix (V):** $V = XW_V where W_V \in R^{d_{model} \times d_v}$

➔ **Query (Q - "What I am looking for"):** This represents the active token trying to gather context. For instance, in the sentence *"The bank of the river"*, the word "bank" casts a Query asking, *"What kind of bank is this? Are there water-related cues nearby?"*

➔ **Key (K - "What I contain / My label"):** This represents every token acting as a candidate index or tag. The word "river" projects a Key that essentially says, *"I am a water-related, geographical feature."* Keys are what the Queries will be matched against.

➔ **Value (V - "The actual substance"):** This represents the actual semantic content or "payload" of the token. Once a Query matches a Key, the corresponding Value is what gets extracted and pulled back into the token's representation.

Q and K specialize in *finding relationships* (calculating compatibility scores via dot products). ***V*** specializes in *passing content* (delivering the updated semantic meaning once a relationship is found).

Scaled Dot-Product Attention Mechanics and Softmax Normalization

Scaled Dot-Product Attention takes the Query (Q), Key (K), and Value (V) matrices we derived earlier and computes a context-aware representation for every token in a single, highly parallelized mathematical operation. formula is:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1: Unnormalized Dot Product Scoring (QK^T)

- To find how much a query token relates to all key tokens, you multiply Q by the transpose of K (K^T).
- **The Result:** For a batch of sequences, this produces a similarity matrix of shape $(Batch, N_q, N_k)$. Each cell (i, j) represents the raw compatibility score between query token i and key token j.

2: Variance Scaling ($\frac{1}{\sqrt{d_k}}$)

- As the dimensionality of the vectors (d_k) grows larger (e.g., 64, 128, or more), dot products grow large in magnitude. Large values push the Softmax function into regions with extremely small/vanishing gradients, halting learning.
- **The Fix:** Dividing by the square root of the key dimension ($\sqrt{d_k}$) scales the variance back down to unit variance, stabilizing gradients.

3: Softmax Normalization

- The scaled scores are passed through a Softmax function across the last dimension.
- **The Result:** This converts raw scores into a probability distribution summing to 1.0 across each row. It tells the model precisely *how much attention* a query token should pay to each respective key token.

4: Weighted Value Projection (... V)

- Finally, the attention probability weights are multiplied by the Value matrix V.
- **The Result:** This creates a linear combination of the values. If token 2 has an attention weight of 0.8 with token 5, 80% of token 5's semantic "value payload" is blended into token 2's new context-aware vector.

By executing this, GPUs can compute relationships between *every word in a sentence simultaneously*. Unlike older recurrent neural networks (RNNs) that processed text sequentially word-by-word, Scaled Dot-Product Attention enables massive parallelization, making large language models computationally feasible.

Recap of all prerequisite modules we use in SynapseGraph-RAG

1. **aiohttp + Semaphore + aiosqlite**: Raw text payloads and candidate entity nodes are extracted concurrently from external LLM endpoints and remote graph APIs. `asyncio.Semaphore(...n..)` enforces strict rate limits, while `aiosqlite` caches payloads to local disk asynchronously without blocking the single-threaded event loop.
2. **sentence-transformers/all-MiniLM-L6-v2**: Extracted text chunks are passed to pre-trained transformer embeddings, mapping text into continuous 384-dimensional vectors (R^{384}). Batched vector operations leverage SIMD instructions and GPU acceleration.
3. **NetworkX + AdjacencyList / graphlib.TopologicalSorter**: Relational entity triples are parsed into sparse Adjacency Lists. Node importance is evaluated via Degree Centrality $C_D(v) = \frac{deg(v)}{|V|-1}$, pruning low-scoring leaf nodes to preserve context window budgets. Multi-hop retrieval steps are sequenced using `graphlib.TopologicalSorter`.
4. **Scaled Dot-Product Attention**: Retrieved entity vectors (K, V) and user query vectors (Q) enter the scaled dot-product attention module. Scores are scaled by $1/\sqrt{d_k}$ to prevent gradient saturation, generating context-weighted output embeddings for downstream LLM generation.

Resources used for learning :

★ Asynchronous Programming. Blocking I/O
https://luminousmen.com/post/asynchronous-programming-blocking-and-non-blocking/
★ Blocking vs Non-blocking vs Asynchronous I/O
https://medium.com/codetodeploy/blocking-vs-non-blocking-vs-asynchronous-i-o-e1c3750a1b81
★ Asynchronous, Non-Blocking, Event-Base architectures
https://stackoverflow.com/questions/7931537/whats-the-difference-between-asynchronous-non-blocking-event-base-architectu
★ Chapter 4: Vectors, Matrices, and Linear Algebra Stanford -
https://web.stanford.edu/class/nbio228-01/handouts/Ch4_Linear_Algebra.pdf
★ RAG Under the Hood: Vectors and Cosine Similarity
https://medium.com/@abi.shree279/rag-under-the-hood-vectors-and-cosine-similarity-f4c8af0b2a59
★ Traversal — NetworkX 3.7 documentation
https://networkx.org/documentation/stable/reference/algorithms/traversal.html
★ A Beginner's Guide to Tokens, Vectors, and Embeddings -
https://medium.com/@saschametzger/what-are-tokens-vectors-and-embeddings-how-do-you-create-them-e2a3e698e037
★ Scaled Dot-Product Attention Explained
https://apxml.com/courses/foundations-transformers-architecture/chapter-2-attention-mechanism-core-concepts/scaled-dot-product-attention
★ Understanding Scaled Dot-Product Attention in Transformers -
https://ai.plainenglish.io/understanding-scaled-dot-product-attention-in-transformers-8cd794ed8164